Blockchain For Developers:
Developer Tools

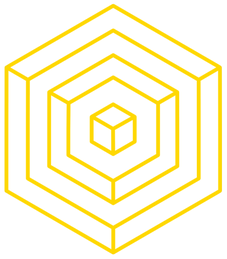
Course Leads:
Sara Magaziotis-Ginori
Olivia Li
Isaac Oh

Aman Shah
Elson Liu

Fujia Wang

BLOCKCHAIN
AT BERKELEY

0 Announcements



Homework #1

- Due today, 11:59pm
- Submission link is on the GitHub and Discord

Since it’s the first week and students are still enrolling, we’ll be lenient on Homework #1’s submission!

Try and get it done by next Wednesday, 11:59pm

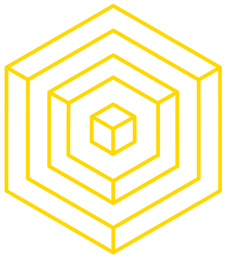
Homework #2

- Due next Wednesday, 11:59pm
- Already uploaded to GitHub
  - Instructions of deploying an NFT to Ethereum

Course GitHub + Discord Channel

https://github.com/BerkeleyBlockchain/fa25-dev-decal

https://discord.gg/QVbNNts3



Starting Anonymous Feedback Forms

- We want honest opinions about the content, speed, our teaching, etc etc
- Anonymous feedback form is up on the GitHub

0.5 Today's Roadmap

What We're Covering Today + Why

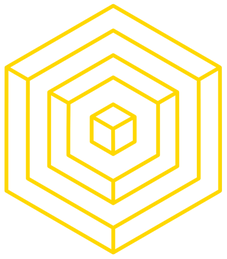
- What: Tools for building on Ethereum
- Why: Ethereum is a **general-purpose blockchain** platform that goes ***beyond*** simple peer-to-peer payments (like Bitcoin)
  - Ethereum introduced Smart Contracts
    - Smart Contracts - self executing programs that run on the blockchain when certain conditions are met
  - Ethereum Virtual Machine (EVM)
    - Global, decentralized computer that executes code in a secure and deterministic way
  - Ethereum developer ecosystem
    - Largest community of blockchain developers are developing on Ethereum!

AUTHOR: Sara Magaziotis-Ginori

Q: What are some problems with deterministic fully public code?

Problems with code on chain

- Front-running and MEV, security risks
  - Attackers can exploit transactions and problems in the code
- Lack of privacy
  - Personal data
- Deterministic code:
  - Everytime you run the code, you need to get the same result, can be hard to bring in off chain data like weather, time, randomness
    - Use oracles, ie Chainlink
- Competitive disadvantage, intellectual properties
- Scalability
  - Deterministic, every node runs code, expensive gas fees. We will talk abt how to optimize
- Updating code and adding features
  - Immutability, cannot patch code. You have to redeploy the contract
  - Old versions remain on chain!!



LECTURE OVERVIEW

1. DEVELOPMENT TOOLS OVERVIEW
2. METAMASK & ETHERSCAN
3. REMIX IDE
4. TESTNETS & RPC
5. FOUNDRY
6. SOLIDITY BASICS

1 DEVELOPMENT TOOLS

DEVELOPMENT TOOLS

METAMASK

REMIX

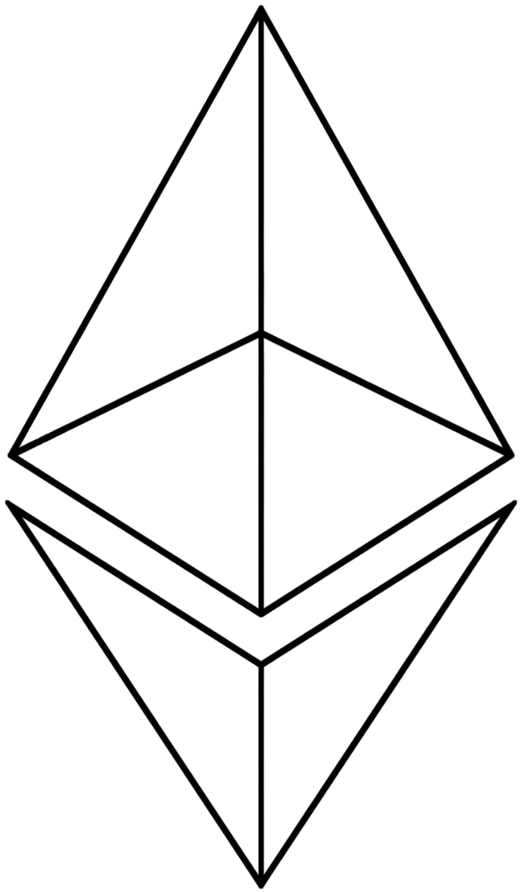
ETHERSCAN

FOUNDRY

AUTHOR: MINXING CHEN

DEVELOPMENT TOOLS

...AND GOOD



Mist Blockchain Browser

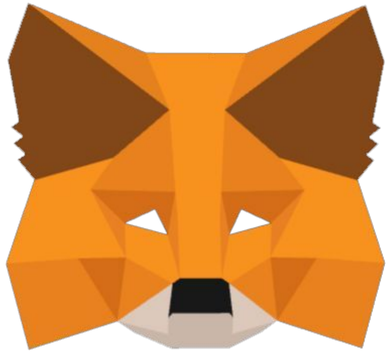
HOMEBREW

Essential macOS package manager

GETH

The Go-Ethereum client

```
brew tap ethereum/ethereum
brew install ethereum
```



METAMASK*

Browser extension; acts as interface to Dapps

SOLC*

Solidity compiler

```
brew install solidity
```

DEVELOPMENT TOOLS

...AND GOOD

NODE.JS

A useful JavaScript library

NPM

JavaScript package manager

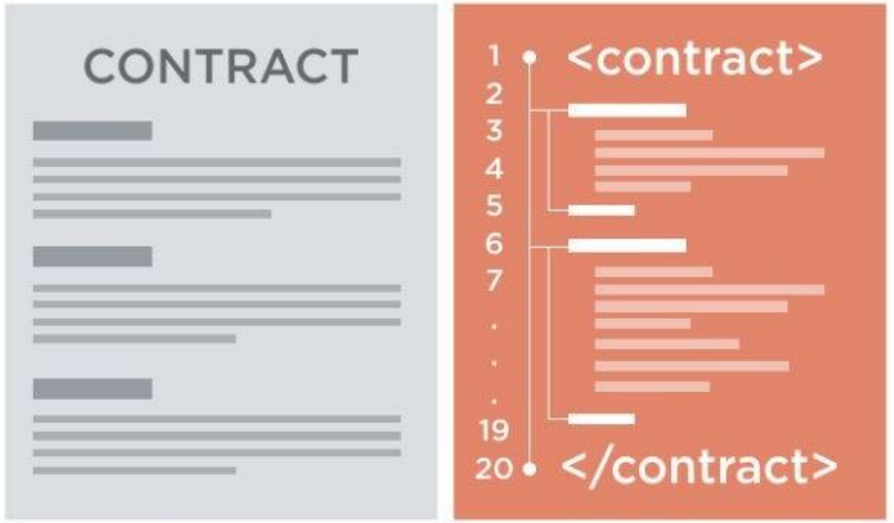
TRUFFLE

Incredibly useful development environment and testing framework

GANACHE/TESTRPC

Simulation of full client behavior

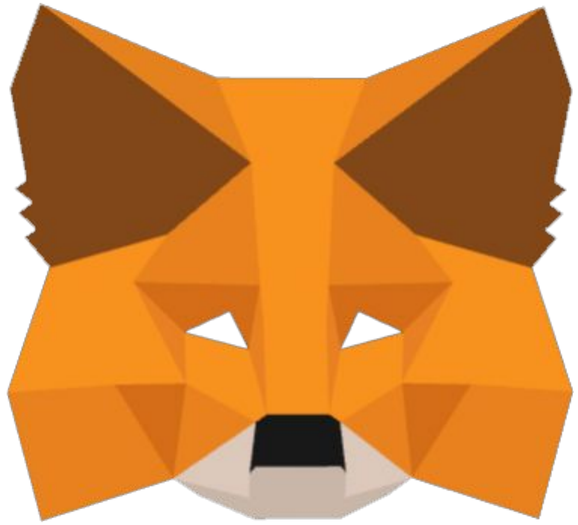
TRUFFLE



2 METAMASK & ETHERSCAN

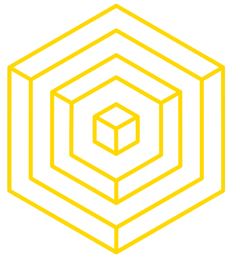
DEVELOPMENT TOOLS

THE MEANS TO ALL EVIL



- Chrome extension that allows access to Ethereum-based dApps in a browser
  - Creation and management of identities/accounts
  - Wallet functionality: interface for accounts and transactions
  - there are other wallets like Phantom, Coinbase

AUTHOR: HAENA LEE



ETHERSCAN Demo

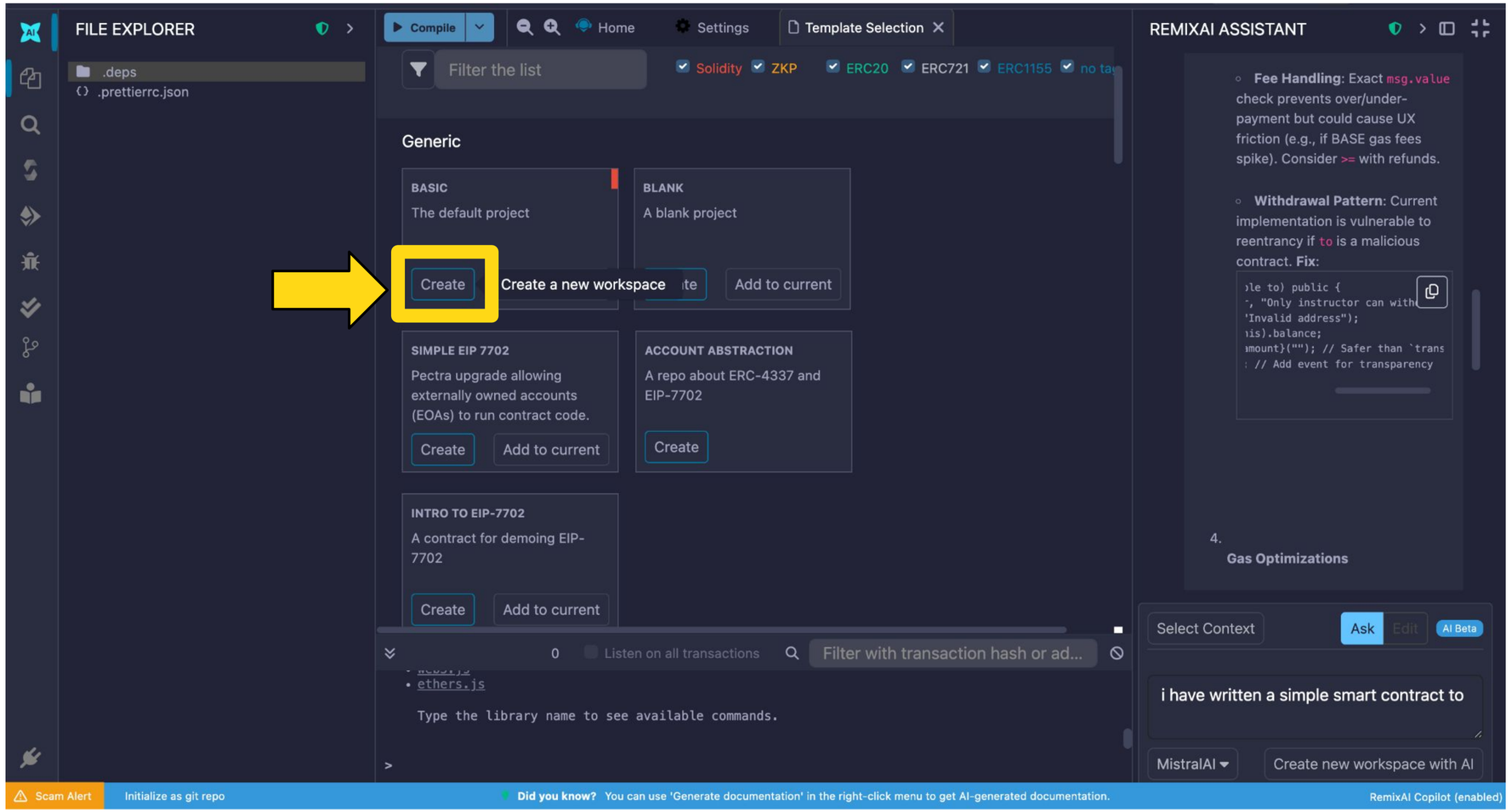
- sign into my wallet
- get public wallet address
- past it onto etherscan
- everyone can see my past activity and balance

3 REMIX IDE

```
pragma solidity >=0.7.0 <0.9.0;
import "remix_tests.sol"; // this import is automatically injected by Remix.
import "hardhat/console.sol";
import "../contracts/attendance.sol";


contract AttendanceTest {
    Attendance attendance;

    /// Called before each test
    function beforeEach() public {
        attendance = new Attendance();
    }

    /// Test: contract starts with zero students
    function testInitialStudentCountIsZero() public {
        Assert.equal(attendance.getNumberOfStudents(), uint(0), "Initial student count should be zero");
    }

    /// Test: add one student
    function testAddSingleStudent() public {
        attendance.addStudent("Alice");

        uint count = attendance.getNumberOfStudents();
        Assert.equal(count, uint(1), "Student count should be 1 after adding Alice");

        string[] memory names = attendance.getStudentNames();
        Assert.equal(names.length, uint(1), "Names array length should be 1");
        Assert.equal(names[0], "Alice", "First student should be Alice");
    }

    /// Test: add multiple students
    function testAddMultipleStudents() public {
        attendance.addStudent("Alice");
        attendance.addStudent("Bob");
        attendance.addStudent("Charlie");

        uint256 count = attendance.getNumberOfStudents();
        Assert.equal(count, uint(3), "Student count should be 3 after adding 3 students");

        string[] memory names = attendance.getStudentNames();
        Assert.equal(names[0], "Alice", "First student should be Alice");
        Assert.equal(names[1], "Bob", "Second student should be Bob");
        Assert.equal(names[2], "Charlie", "Third student should be Charlie");
    }
}
```

REMIX IDE: Template Project

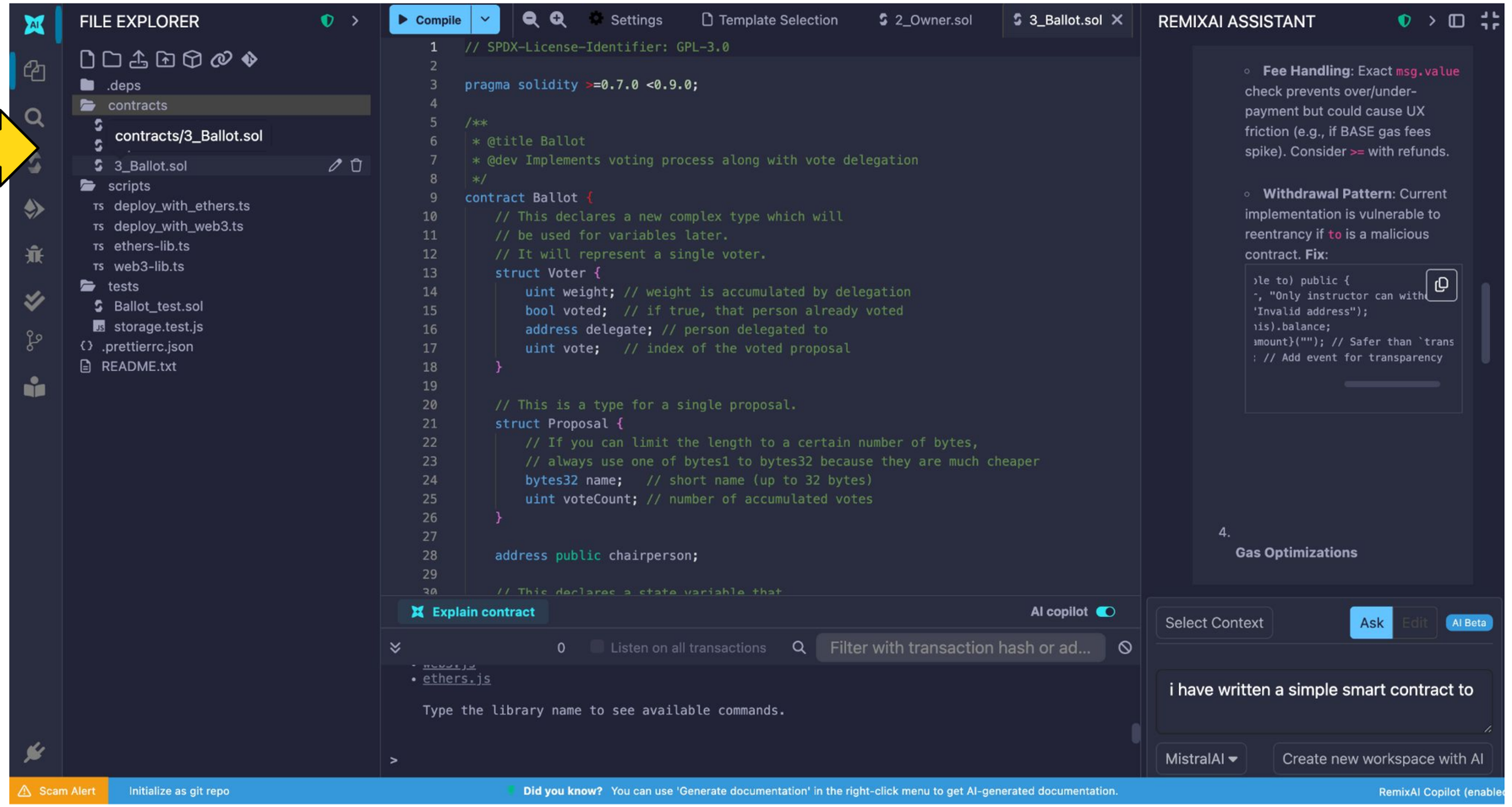
Create a basic project with necessary file structure



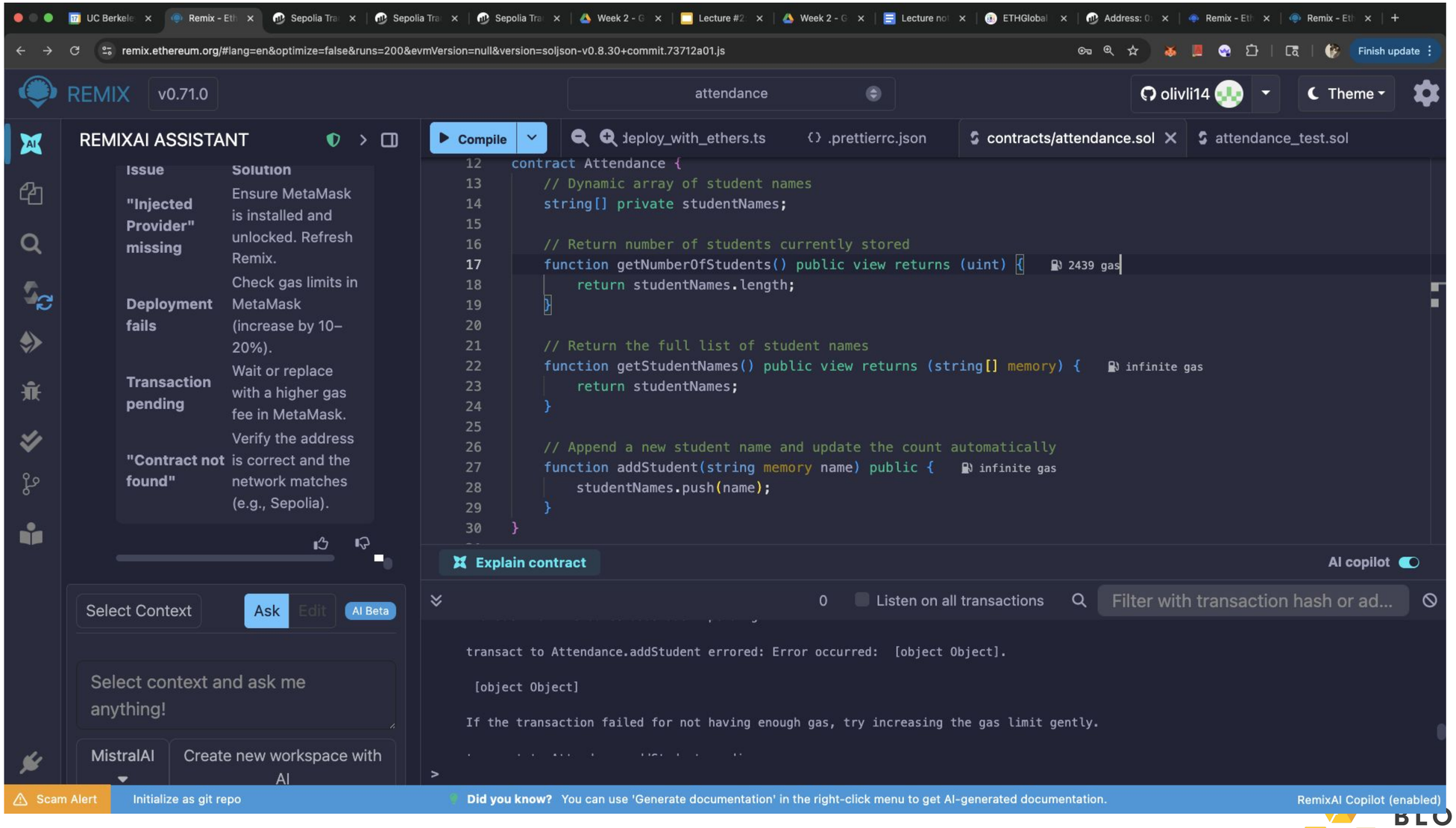
AUTHOR:OLIVIA LI

Write your contracts

.sol is a solidity file

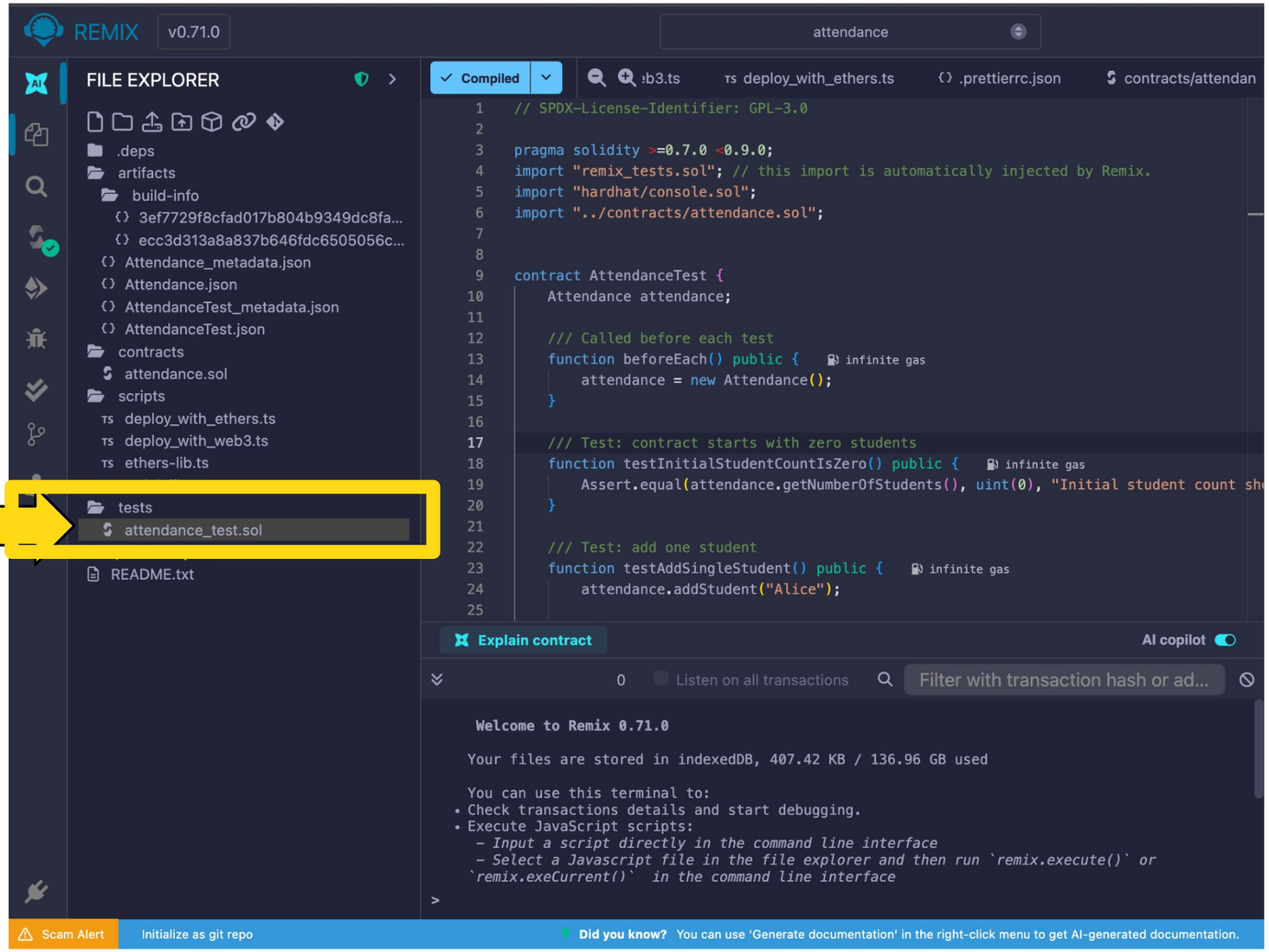


AUTHOR:OLIVIA LI


remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.30+commit.73712a01.js
REMIX
v0.71.0
attendance
olivli14
Theme
REMIXAI ASSISTANT
Issue
Solution
"Injected Provider" missing
Ensure MetaMask is installed and unlocked. Refresh Remix.
Deployment fails
Check gas limits in MetaMask (increase by 10–20%).
Transaction pending
Wait or replace with a higher gas fee in MetaMask.
"Contract not found"
Verify the address is correct and the network matches (e.g., Sepolia).
Select Context
Ask
Edit
AI Beta
Select context and ask me anything!
MistralAI
Create new workspace with AI
Compile
deploy_with_ethers.ts
.prettierrc.json
contracts/attendance.sol
attendance_test.sol
contract Attendance {
// Dynamic array of student names
string[] private studentNames;
// Return number of students currently stored
function getNumberOfStudents() public view returns (uint) {
2439 gas
return studentNames.length;
}
// Return the full list of student names
function getStudentNames() public view returns (string[] memory) {
infinite gas
return studentNames;
}
// Append a new student name and update the count automatically
function addStudent(string memory name) public {
infinite gas
studentNames.push(name);
}
}
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or ad...
transact to Attendance.addStudent errored: Error occurred: [object Object].
[object Object]
If the transaction failed for not having enough gas, try increasing the gas limit gently.
Scam Alert
Initialize as git repo
Did you know? You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.
RemixAI Copilot (enabled)

Test your contracts

write tests in test folder, test before deploying

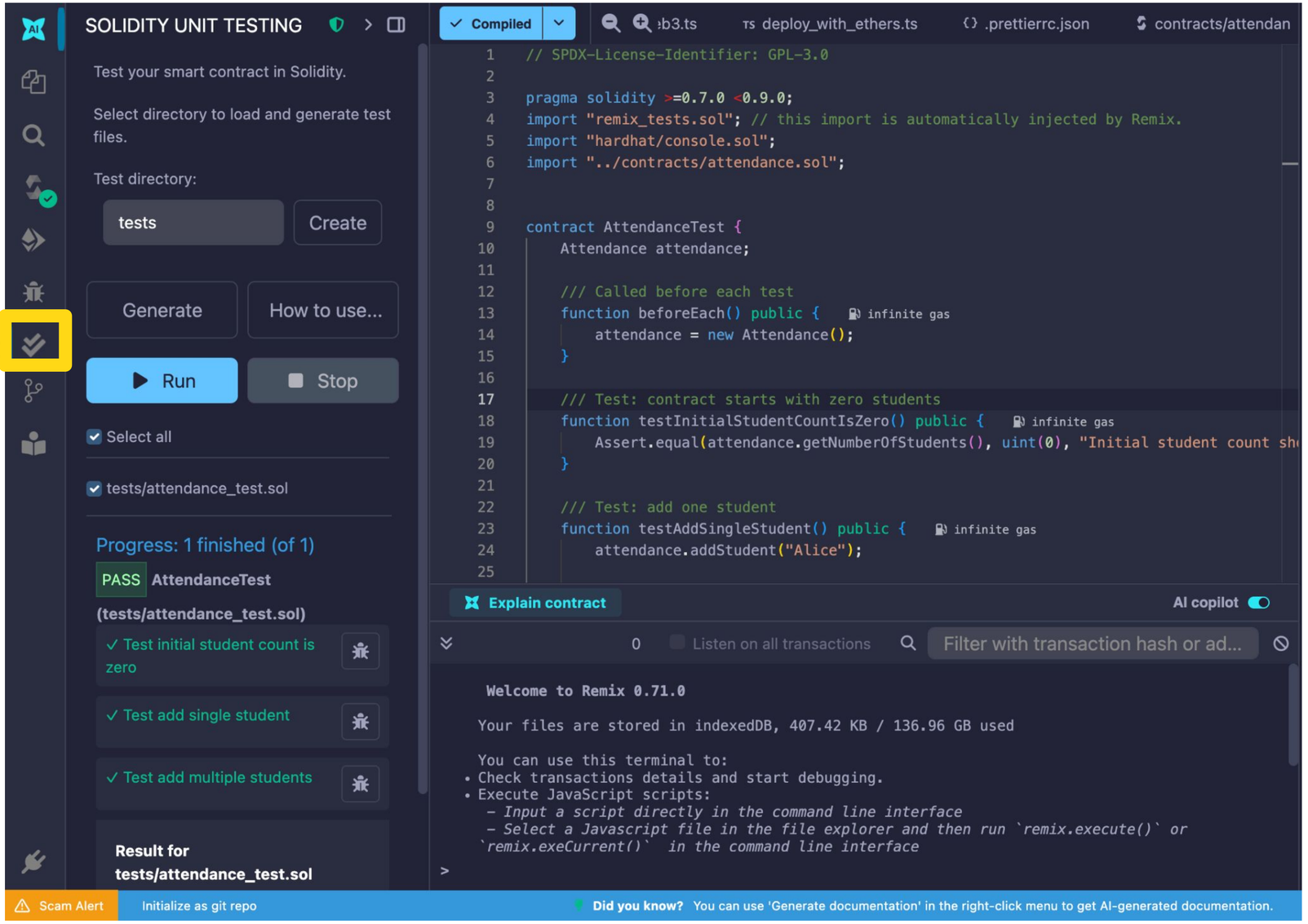


AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

Testing your contracts

Press the compile button. Go to Solidity Unit Testing. Run

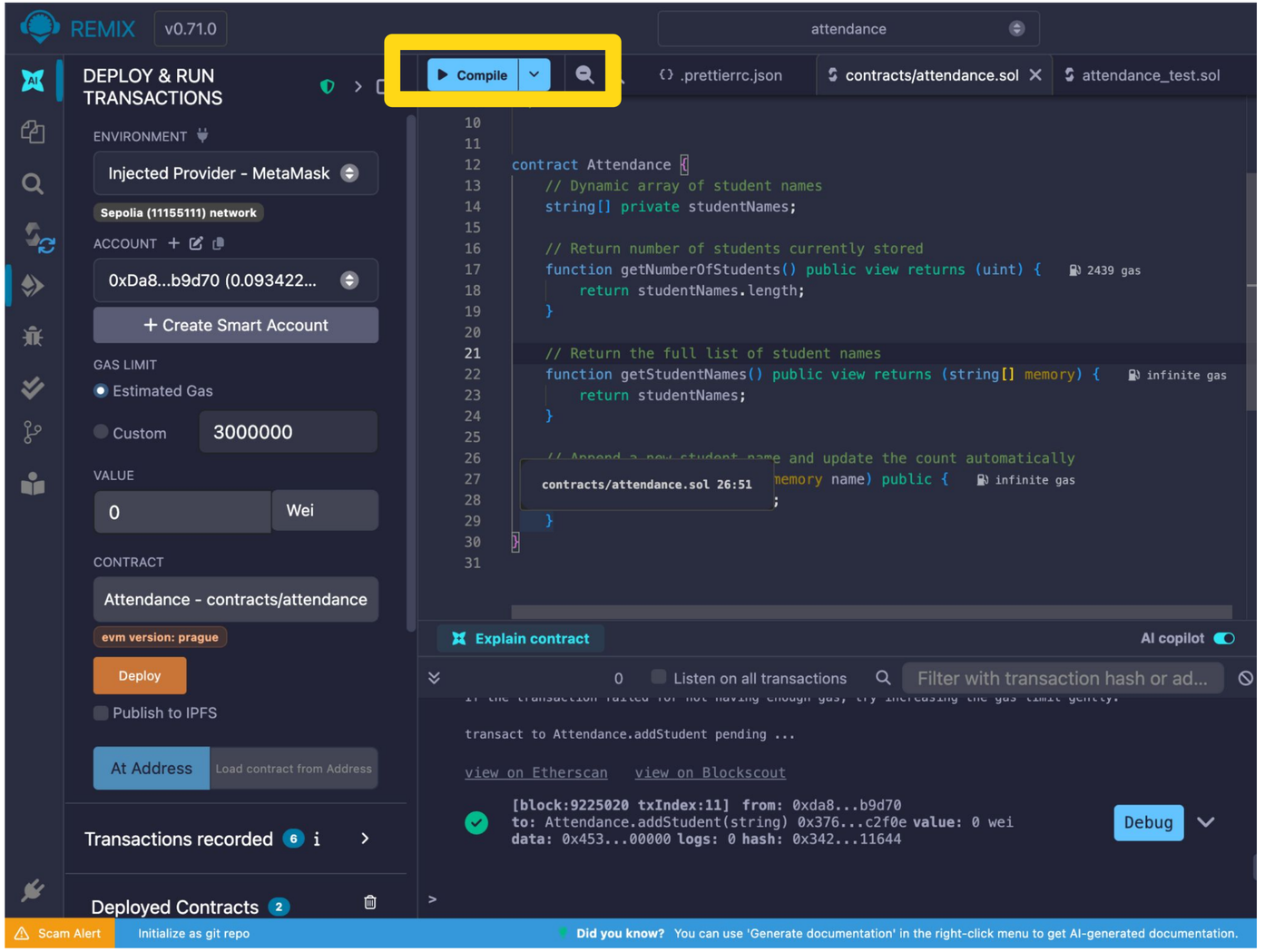


AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

Compile Your Contract

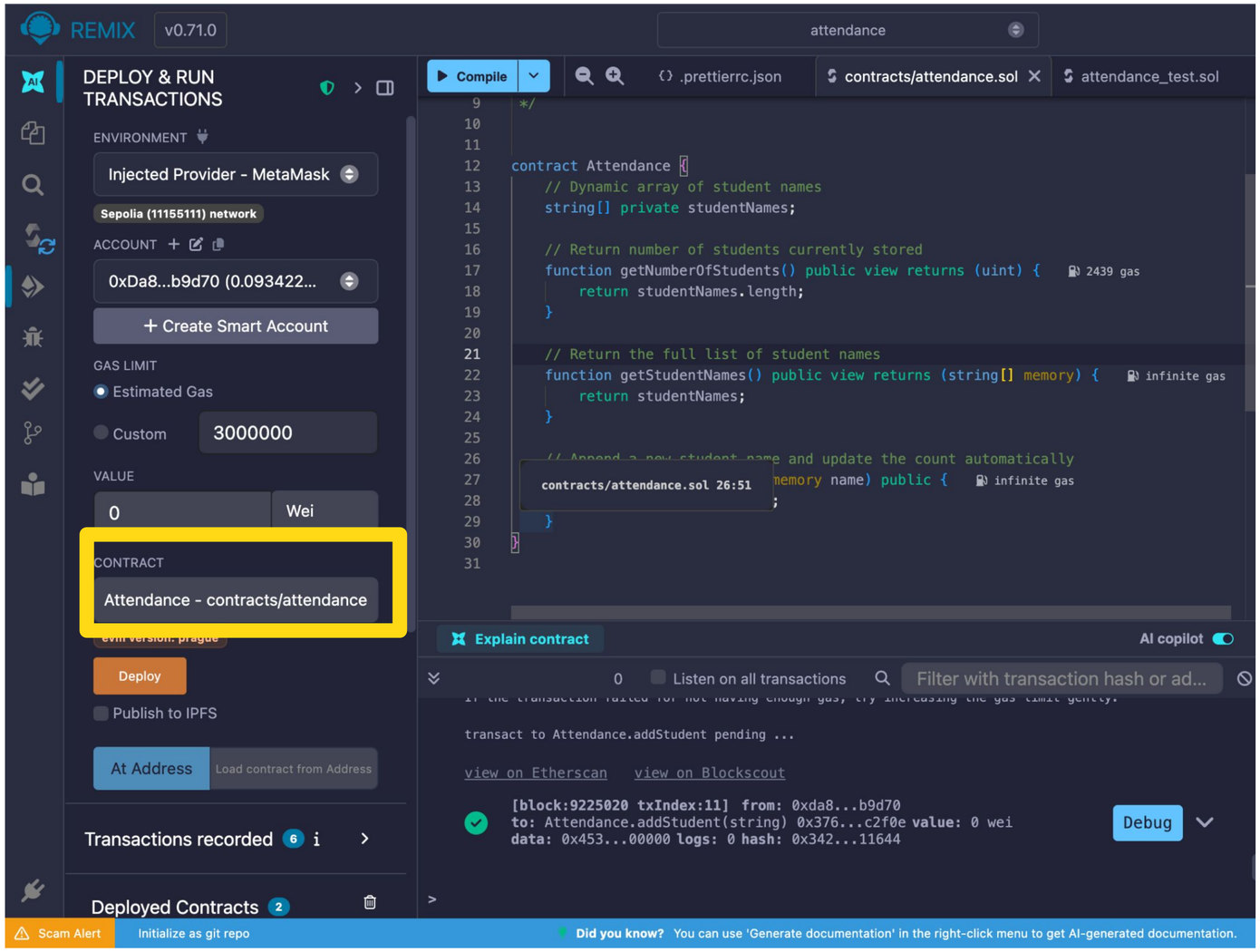
press compile



AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

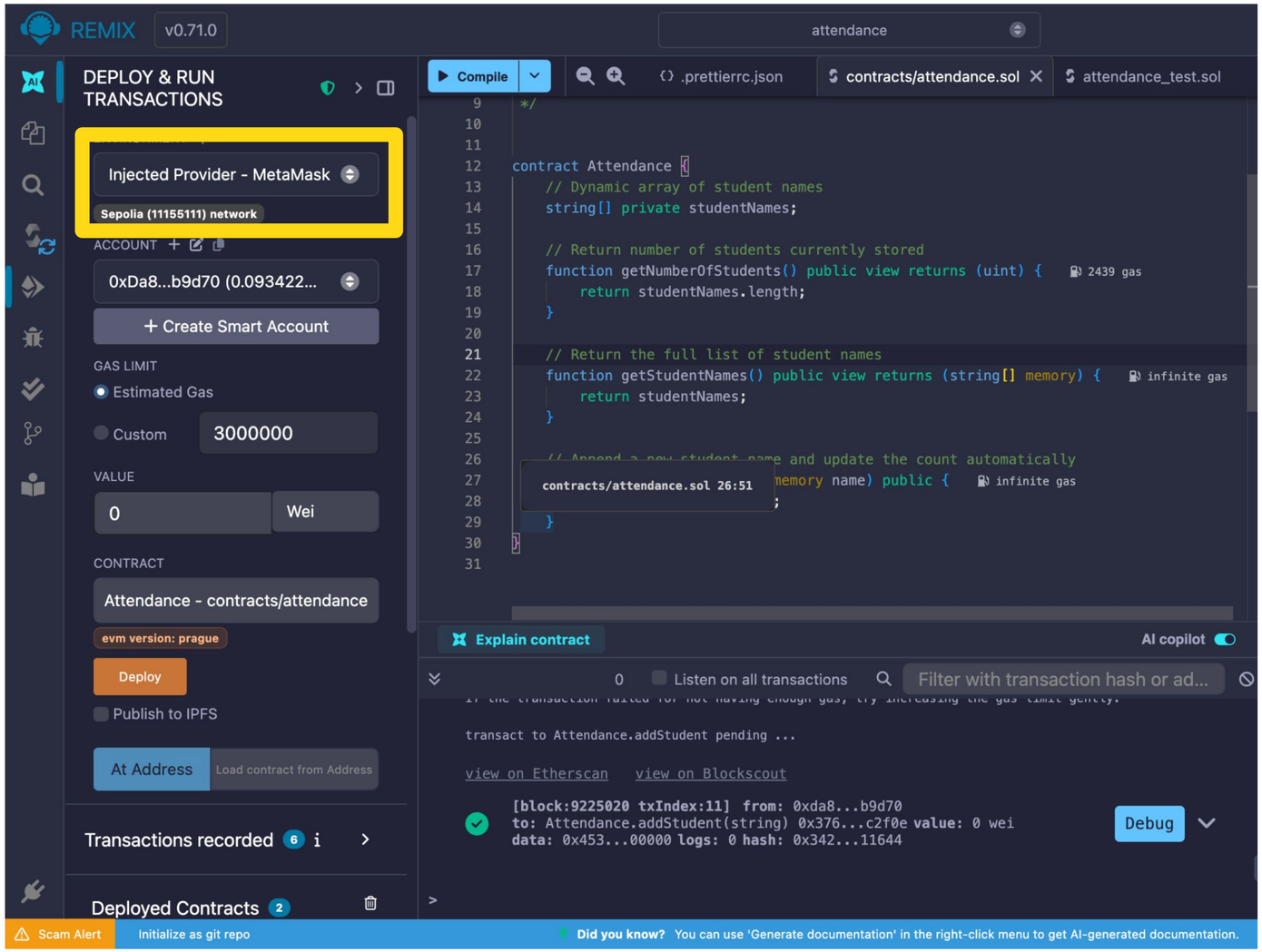
Select the Right Contract



AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

Choose Metamask Provider

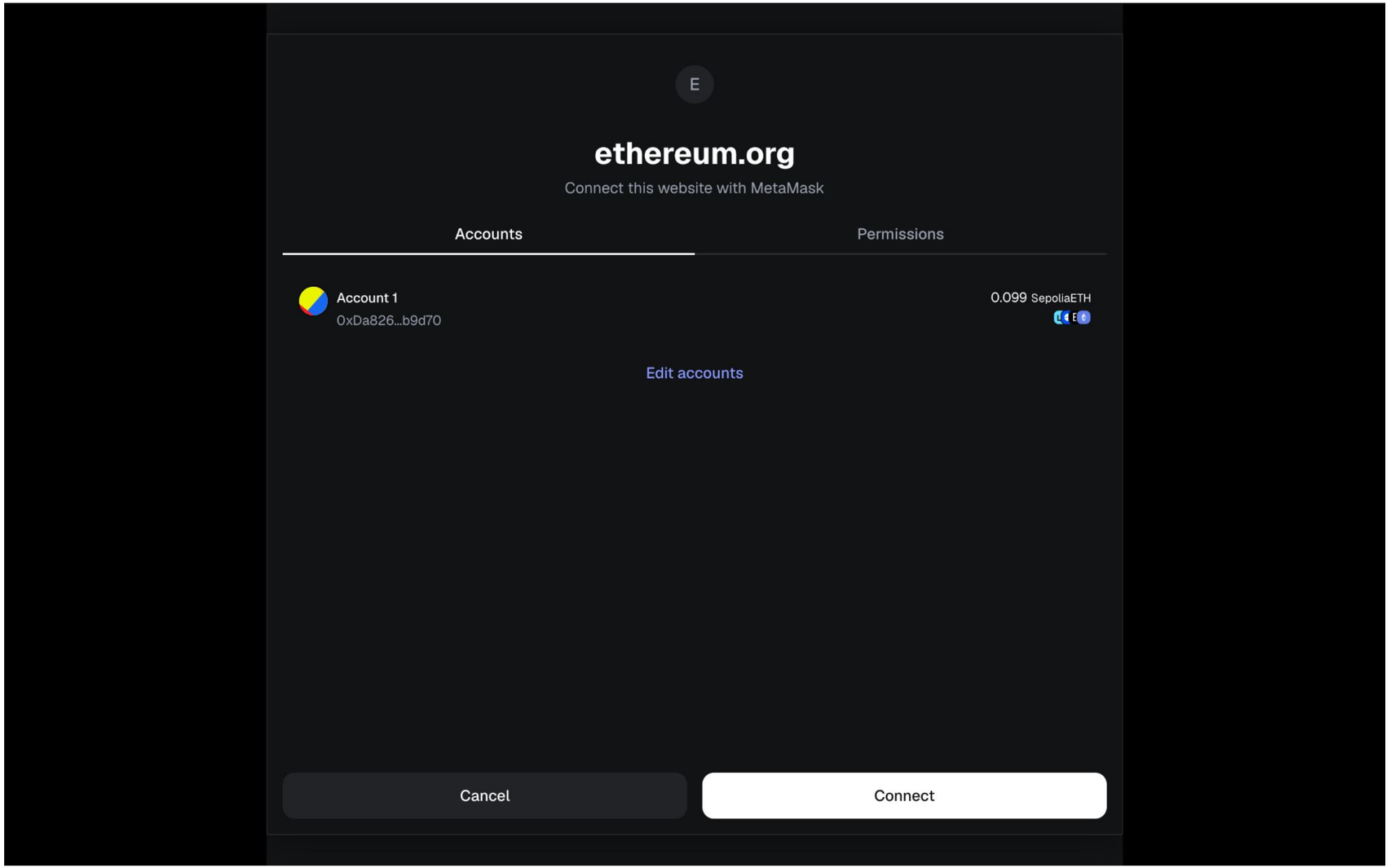


Deploy on Sepolia!!!!

Sepolia is a testnet aka not real money. We don't want to spend real money testing!!

AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

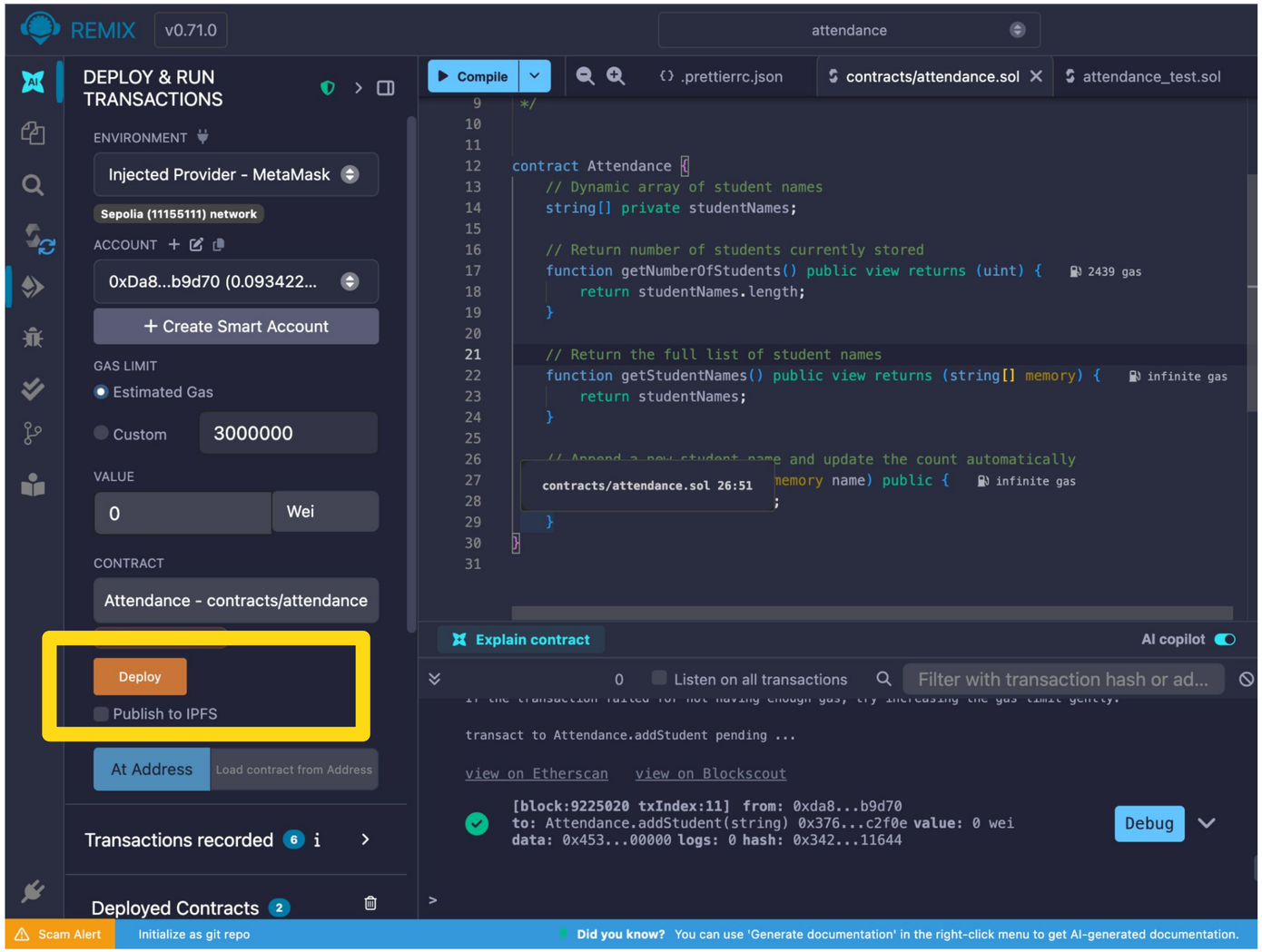


AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

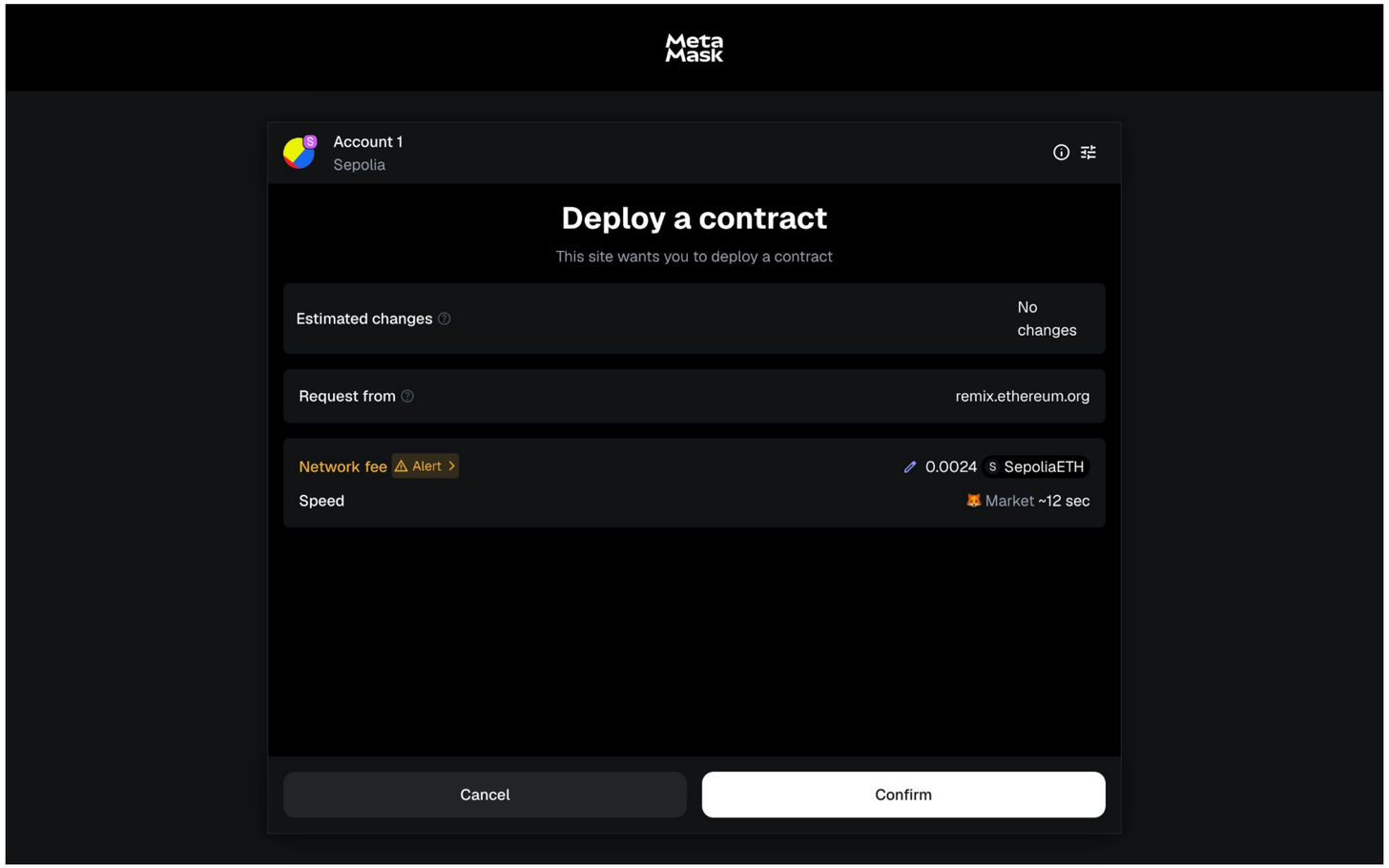
Deploy

put code on blockchain



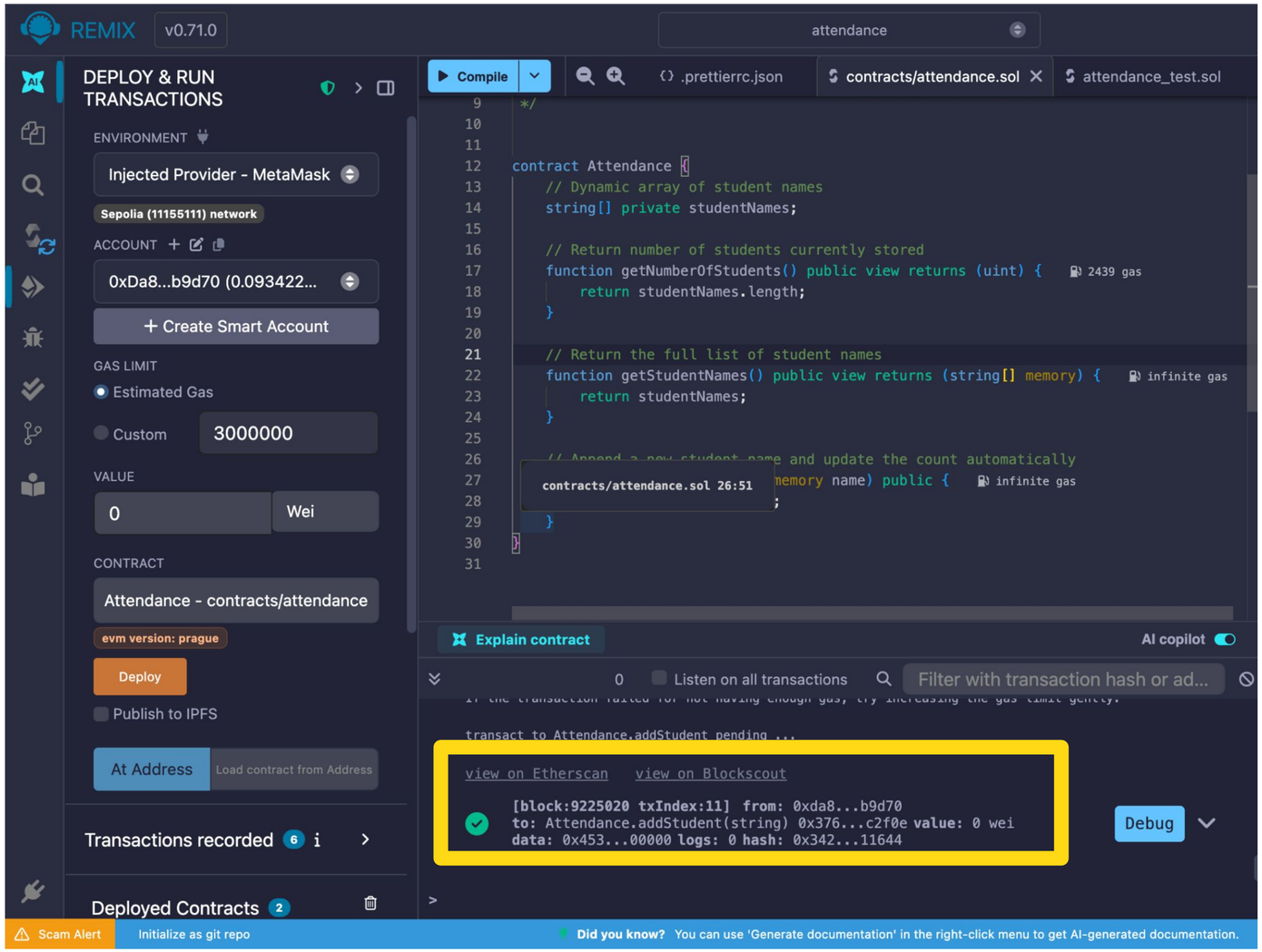
AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY



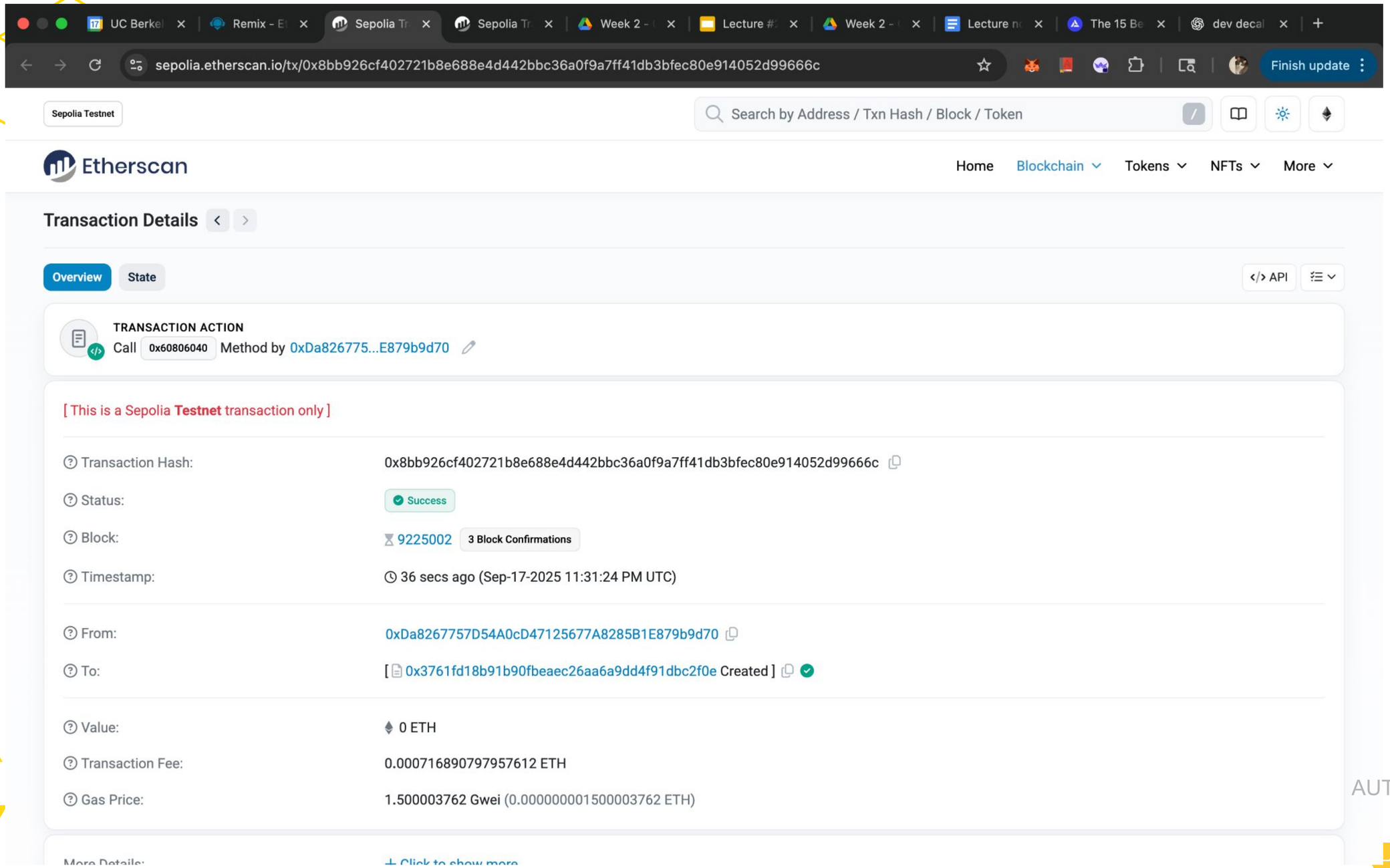
AUTHOR:OLIVIA LI

Look at deployment on Etherscan



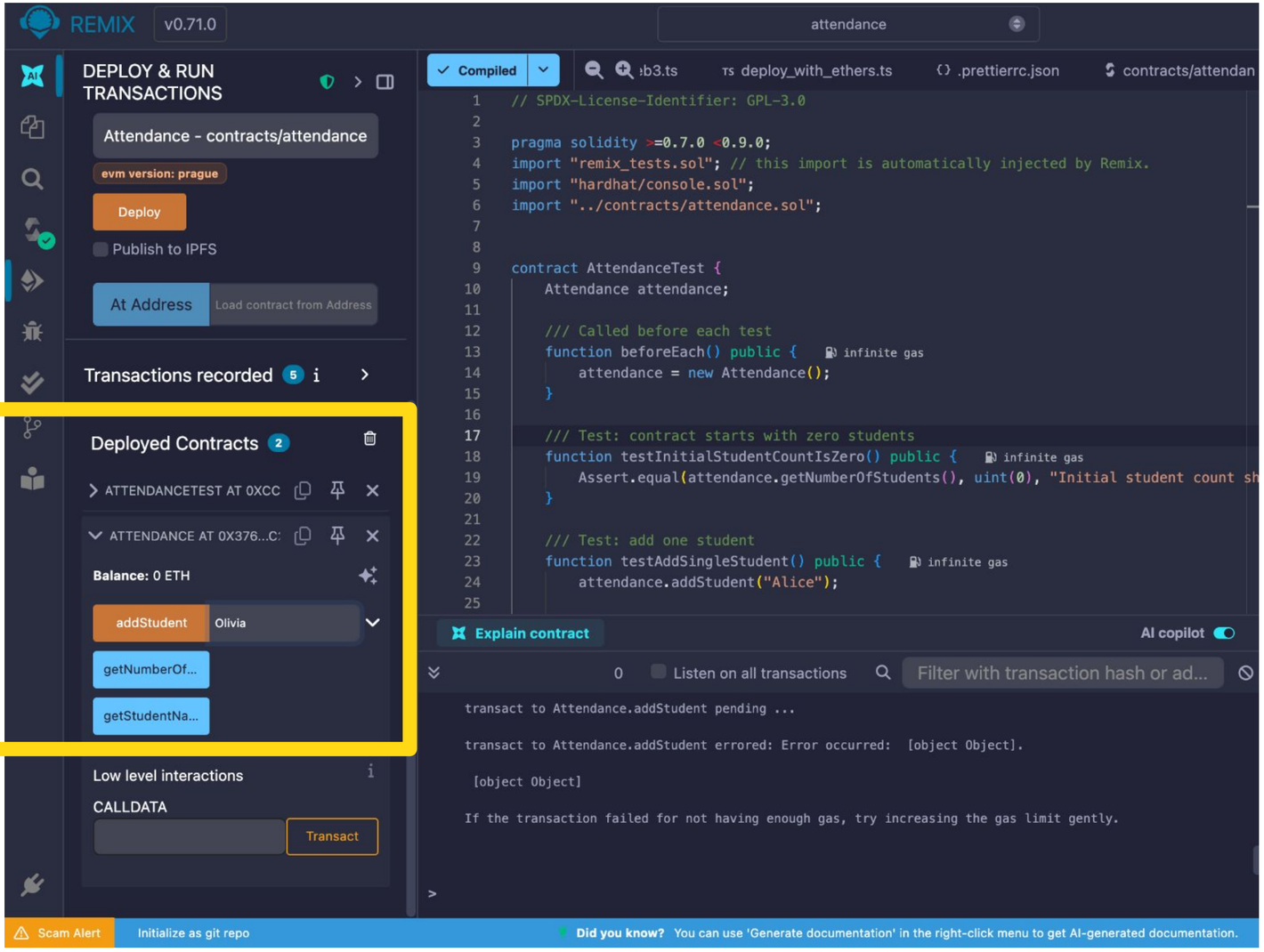
AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY



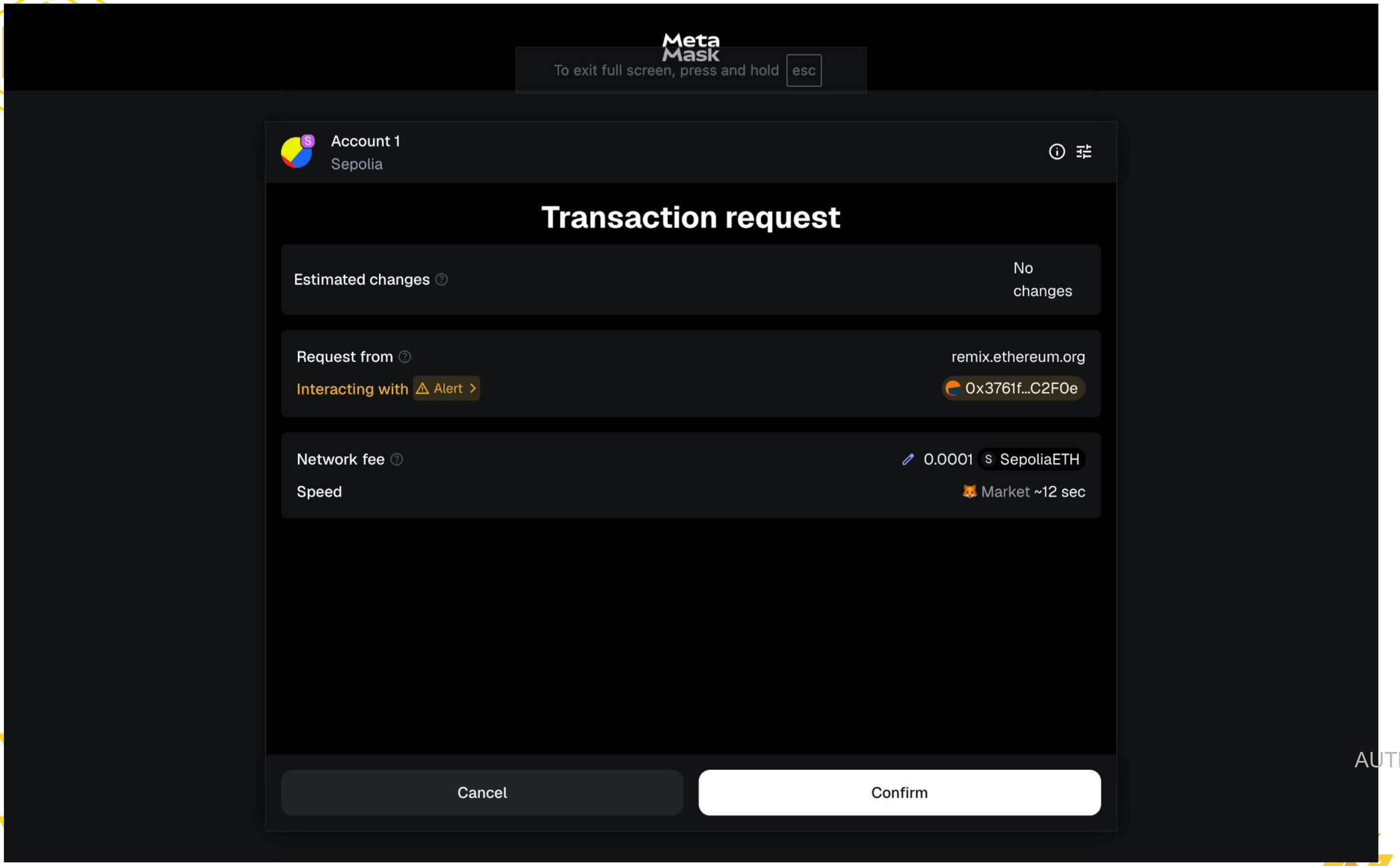
AUTHOR:OLIVIA LI

Test Functions



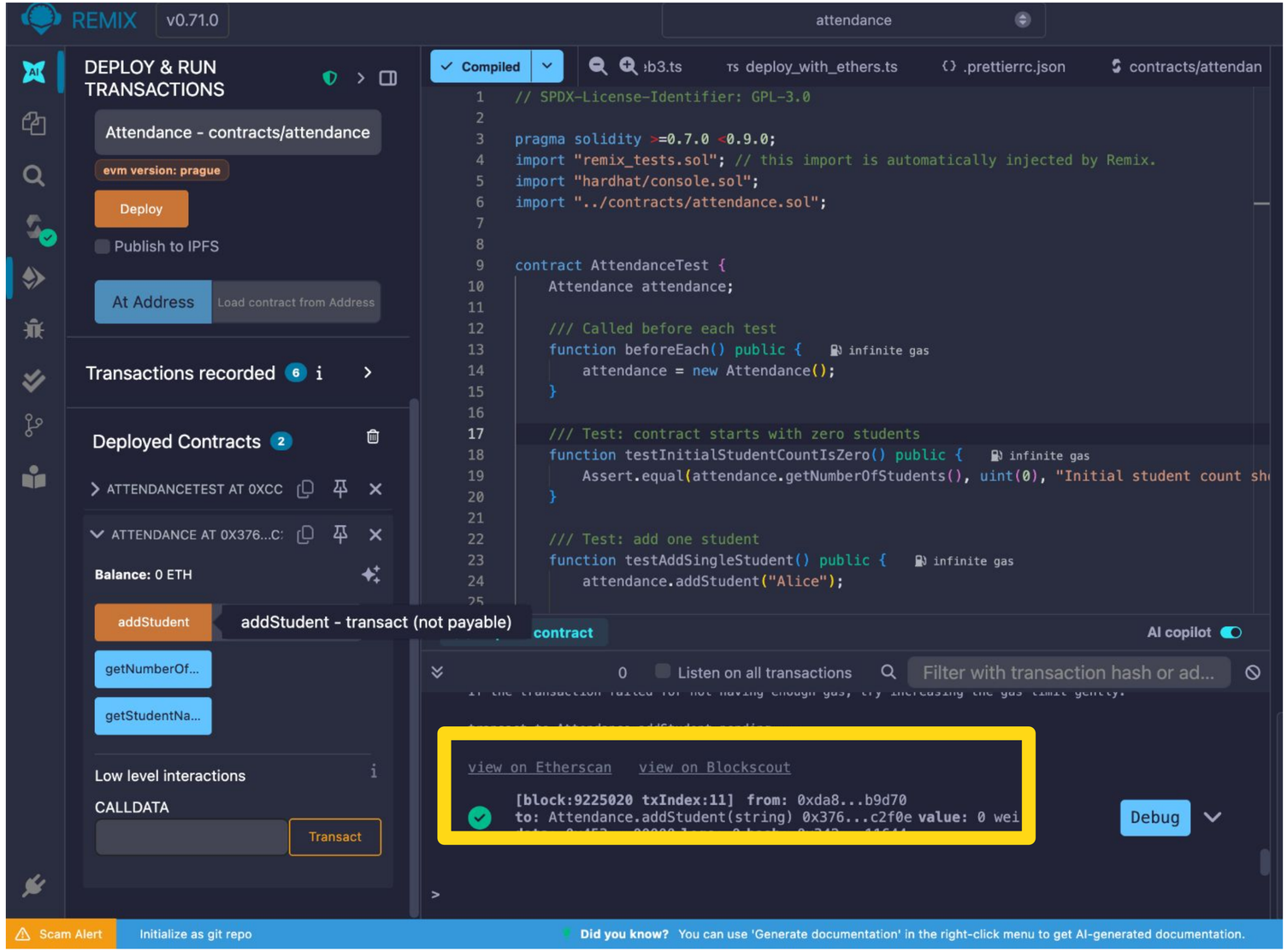
AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY



AUTHOR:OLIVIA LI

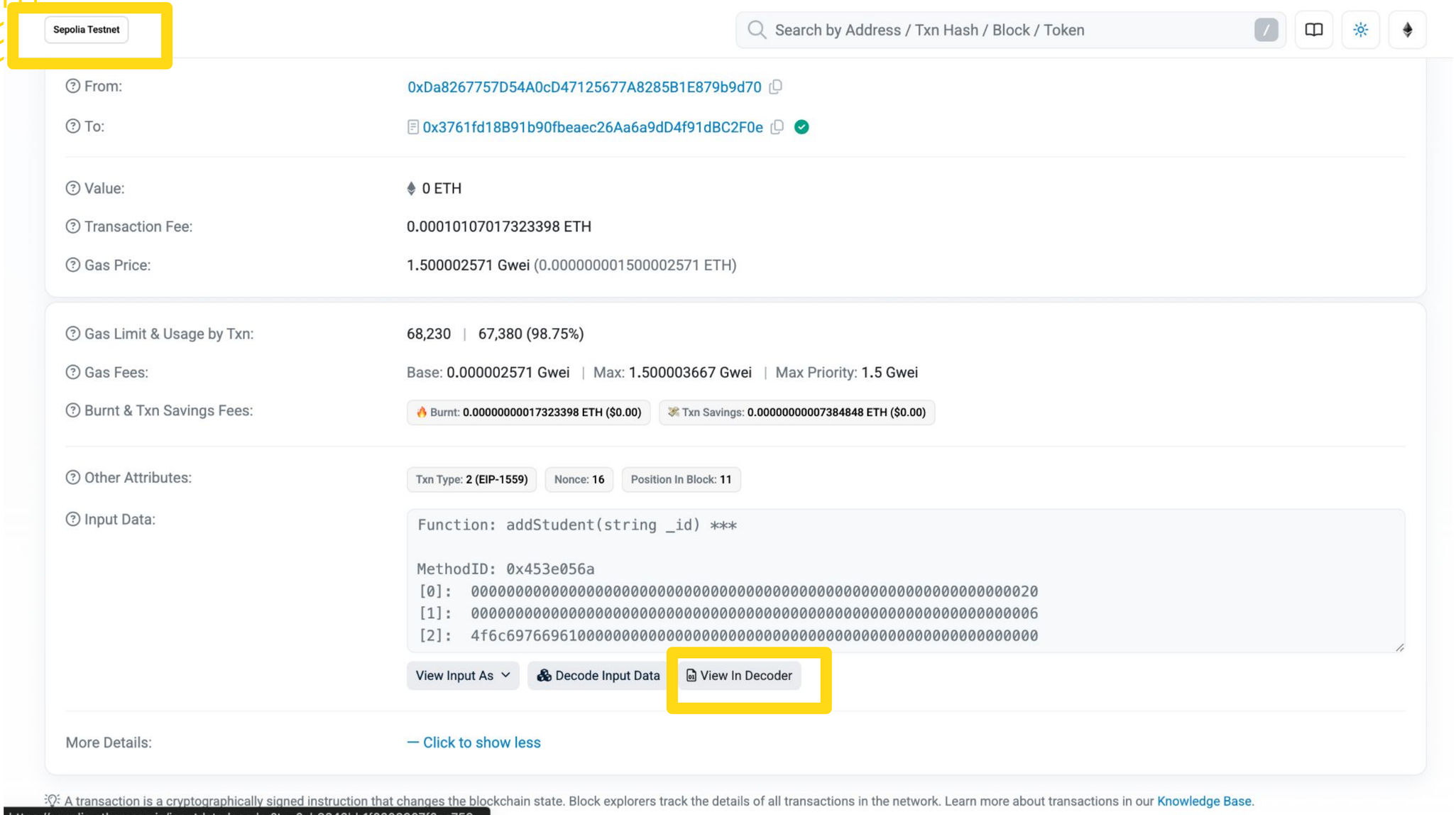
Look at transaction on Etherscan



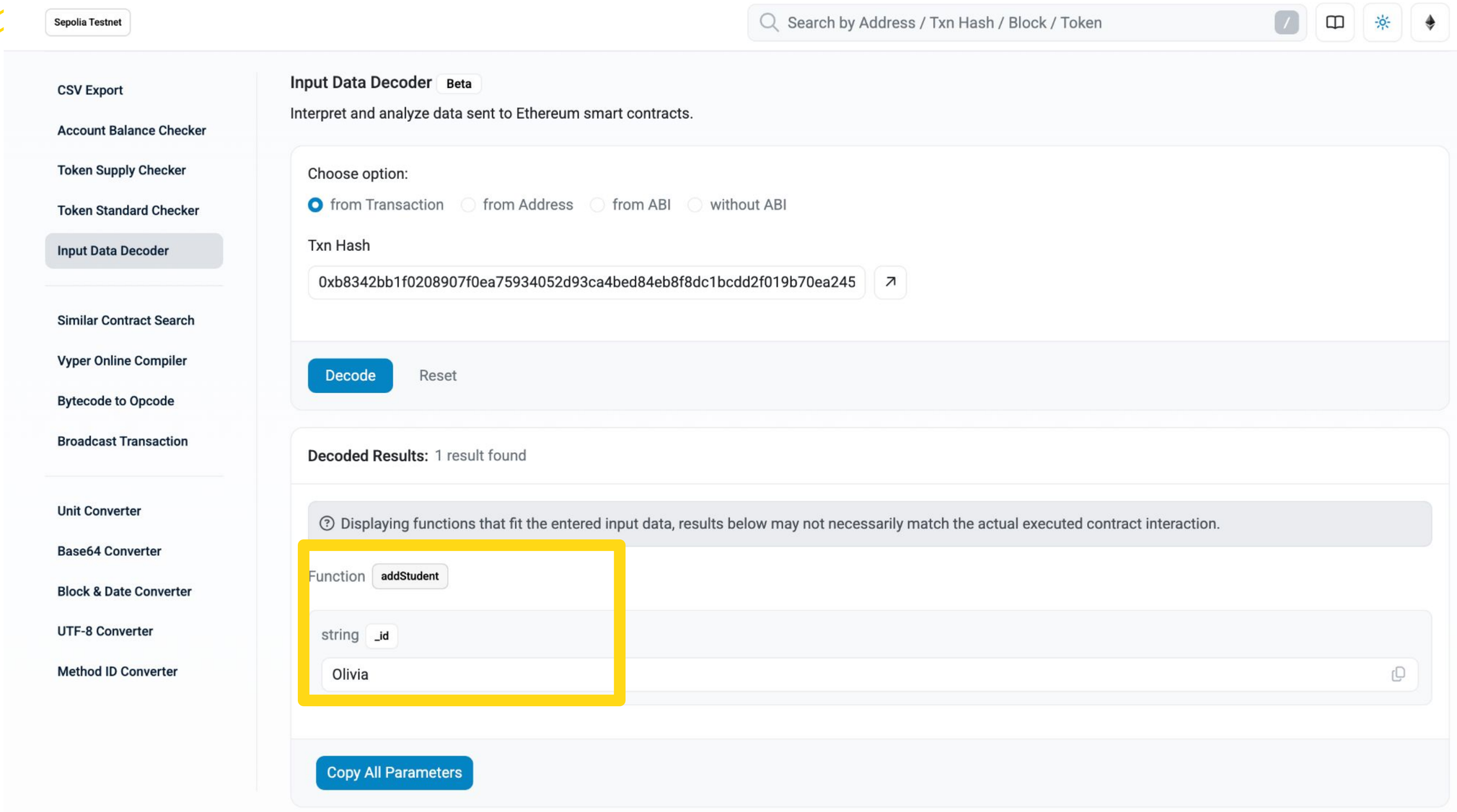
AUTHOR:OLIVIA LI

BLOCKCHAIN AT BERKELEY

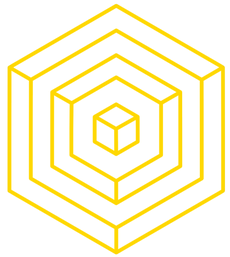
Sepolia Testnet



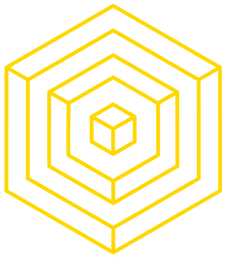
Look at Functino called and input



DEMO TIME!



3 TESTNETS & RPC



TESTNETS

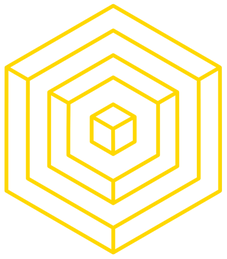
ALTERNATE REALITIES

- What problems might we run up against when trying to test our contract on Ethereum?

TESTNETS

ALTERNATE REALITIES

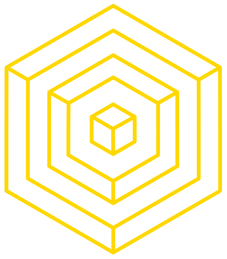
- What problems might we run up against when trying to test our contract on Ethereum?
  - **Immutability**: Bugs and mistakes cannot be changed once we deploy the contract.
  - **Gas Fees**: Transactions cost real-world money, which would make it expensive to test our contracts



TESTNETS
ALTERNATE REALITIES

The solution to these problems are **testnets**.

- A testnet is an alternative instance of a blockchain that is solely used for development.
  - Ethereum testnets run the EVM, smart contracts work exactly the same
  - The tokens (including the native asset such as ETH) have no value
    - “Faucets” give out ETH and other tokens for free



TESTNETS

ALTERNATE REALITIES

- Another option is to **test locally** by using a **local chain**
- Tools such as Foundry and HardHat let you spin up a local blockchain, providing you with public/private keypairs loaded with ETH for you to use.
- Local chains can even be forks of live chains such as Ethereum so that you can access existing contracts

REMOTE PROCEDURAL CALLS

TELLING BLOCKCHAINS TO DO STUFF

- **RPC** stands for:
  - **Remote**
  - **Procedural**
  - **Call**

- RPC calls are used to tell another computer to do something

- To deploy a smart contract, you must submit a transaction to a node, which will then submit it to the mempool. This node can either be:
  - Your own node running on your computer
  - A node running on another computer

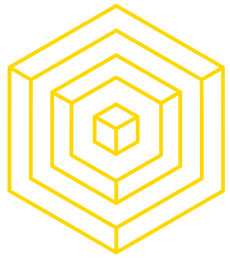
RPC PROVIDERS

TELLING BLOCKCHAINS TO DO STUFF

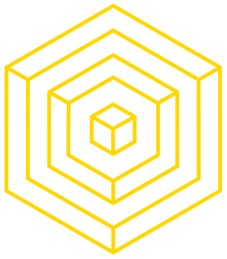
- Companies that provide access to their node for you to interact with the blockchain are called **RPC Providers**

- Examples of RPC Providers for Ethereum:
  - Infura
  - Alchemy
  - Etherscan

- RPC Providers generally give you an API key with which you can send requests to their server.



4 SOLIDITY OVERVIEW

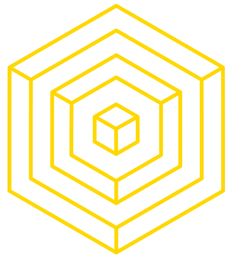


HISTORY OF SOLIDITY

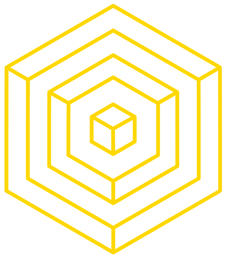
- When Ethereum was proposed in 2013, the vision was a “world computer” where anyone could deploy decentralized programs (smart contracts). Known as the EVM!

- The EVM was designed as the runtime environment for these programs. But the EVM only understands bytecode — raw instructions.

- To make programming easier, developers designed Solidity (around 2014, led by Gavin Wood, Christian Reitwiessner, and others) as a high-level, human-readable language that compiles down to EVM bytecode.



4.1 DATA TYPES

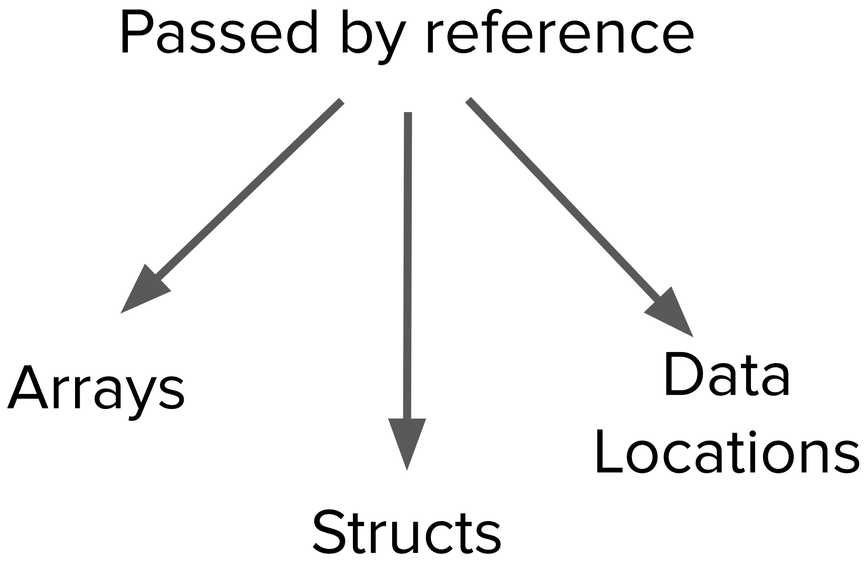


CATEGORIES OF TYPES
DATA TYPES

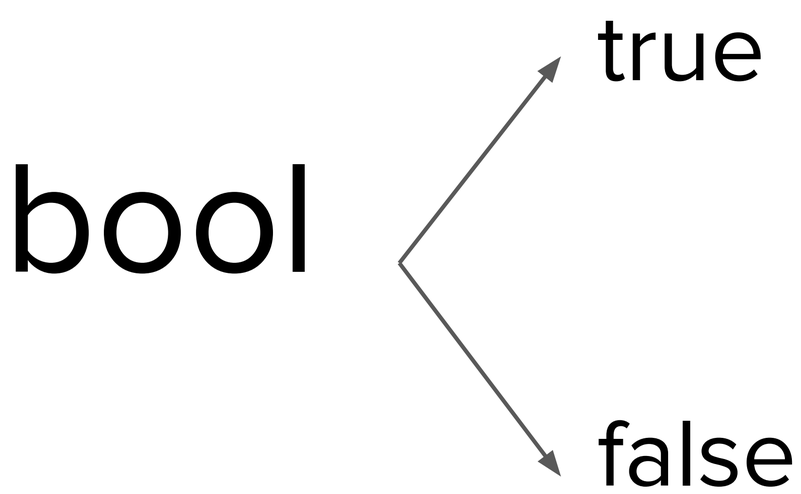
VALUE TYPES

Passed by value

REFERENCE TYPES



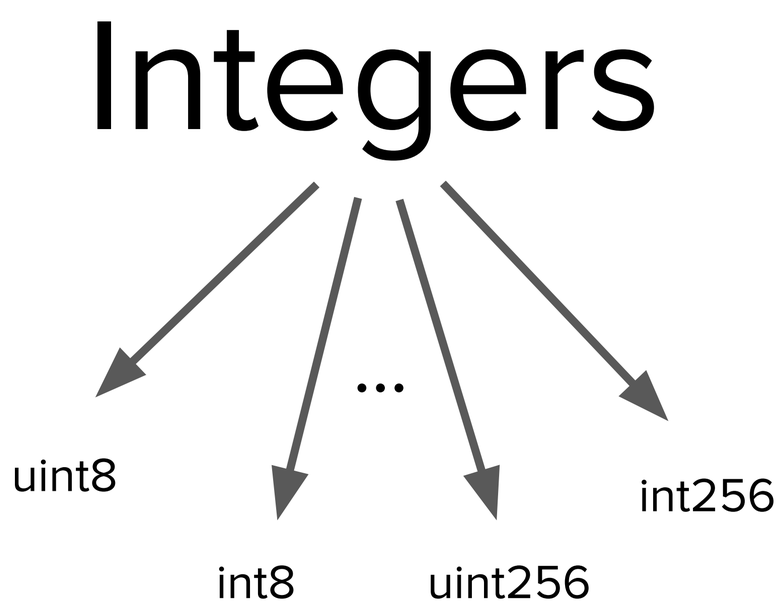
BOOLEANS
VALUE TYPES



Operators:

- ! (logical negation)
- && (logical conjunction, “and”)
- || (logical disjunction, “or”)
- == (equality)
- != (inequality)

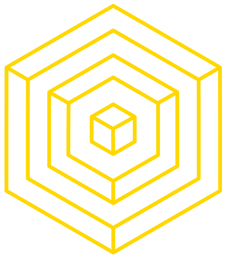
INTEGERS
VALUE TYPES



- int/uint ranges from int8/uint8 to int256/uint256.(In steps of 8)
- Int/uint are alias of int256/uint256

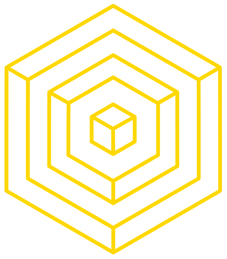
Operators:

- Comparisons: <=, <, ==, !=, >=, > (evaluate to bool)
- Bit operators: &, |, ^ (bitwise exclusive or), ~ (bitwise negation)
- Shift operators: << (left shift), >> (right shift)
- Arithmetic operators: +, -, unary -, *, /, % (modulo), ** (exponentiation)



INTEGERS
VALUE TYPES

Unit	Wei Value	Wei
wei	1 wei	1
Kwei (babbage)	1e3 wei	1,000
Mwei (lovelace)	1e6 wei	1,000,000
Gwei (shannon)	1e9 wei	1,000,000,000
microether (szabo)	1e12 wei	1,000,000,000,000
milliether (finney)	1e15 wei	1,000,000,000,000,000
ether	1e18 wei	1,000,000,000,000,000,000



INTEGERS
VALUE TYPE

Bits Operation
(Performed on Two's Complement)

~int256(0) == int256(-1)

Shifts

x << y == x * 2y**

x >> y == x / 2 ** y

INTEGERS
VALUE TYPE

Addition / Subtraction
(Performed on usual semantics)

uint256(0) - uint256(1) == 2256 - 1**

Multiplication / Division
(Performed on usual semantics)

int256(-5) / int256(2) == int256(-2)

ADDRESS
VALUE TYPE

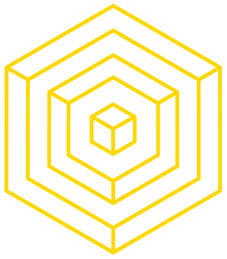
Address & Address Payable

- Address: Holds a 20 byte value (size of an Ethereum address).

- Address Payable: Same as address, but with the additional members transfer and send.

Operators:

```
pragma solidity ^0.5.0;

contract WithdrawalContract {
    address public richest;
    uint public mostSent;

    mapping (address => uint) pendingWithdrawals;

    constructor() public payable {
        richest = msg.sender;
        mostSent = msg.value;
    }

    function becomeRichest() public payable returns (bool) {
        if (msg.value > mostSent) {
            pendingWithdrawals[richest] += msg.value;
            richest = msg.sender;
            mostSent = msg.value;
            return true;
        } else {
            return false;
        }
    }

    function withdraw() public {
        uint amount = pendingWithdrawals[msg.sender];
        // Remember to zero the pending refund before
        // sending to prevent re-entrancy attacks
        pendingWithdrawals[msg.sender] = 0;
        msg.sender.transfer(amount);
    }
}
```



ADDRESS PAYABLE
VALUE TYPE

Transfer & Send

```
address payable x = address(0x123);
address myAddress = address(this);
if (x.balance < 10 &&
myAddress.balance >= 10)
x.transfer(10);
```

The transfer function fails if the balance of the current contract is not large enough or if the Ether transfer is rejected by the receiving account. The transfer function reverts on failure.

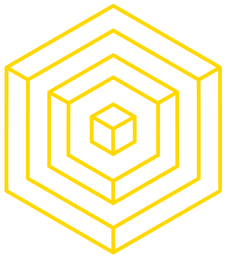
Send is the low-level counterpart of transfer. If the execution fails, the current contract will not stop with an exception, but send will return false.

CONTRACT TYPES

VALUE TYPE

```
pragma solidity ^0.5.0;

contract D {
    uint public x;
    constructor(uint a) public payable {
        x = a;
    }
}

contract C {
    D d = new D(4); // will be executed as part of C's
constructor

    function createD(uint arg) public {
        D newD = new D(arg);
        newD.x();
    }

    function createAndEndowD(uint arg, uint amount)
public payable {
        // Send ether along with the creation
        D newD = (new D).value(amount)(arg);
        newD.x();
    }
}
```

- Contracts do not support any operators.
- The members of contract types are the external functions of the contract including public state variables.
- For a contract C you can use type(C) to access type information about the contract.



STRING LITERALS
VALUE TYPES

bytes32 a = "stringliteral"

string b = "stringliteral"

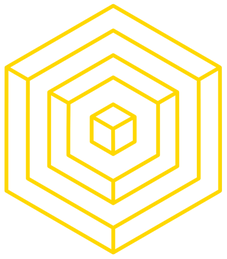
a == b

Hex “DEADBEEF”

"**\n**\"\'\\abc\

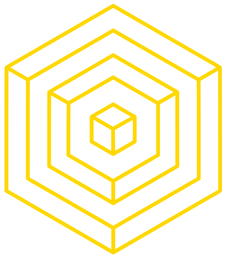
def"

- \<newline> (escapes an actual newline)
- \\ (backslash)
- \' (single quote)
- \" (double quote)
- \b (backspace)
- \f (form feed)
- \n (newline)
- \r (carriage return)
- \t (tab)
- \v (vertical tab)
- \xNN (hex escape))
- \uNNNN (unicode escape))



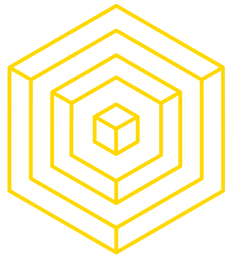
ENUM
VALUE TYPES

```
pragma solidity >=0.4.16 <0.6.0;

contract test {
    enum ActionChoices { GoLeft, GoRight,
GoStraight, SitStill }
    ActionChoices choice;
    ActionChoices constant defaultChoice =
ActionChoices.GoStraight;

    function setGoStraight() public {
        choice = ActionChoices.GoStraight;
    }

    // Since enum types are not part of the ABI, the signature of
"getChoice"
    // will automatically be changed to "getChoice() returns (uint8)"
    // for all matters external to Solidity. The integer type used is
just
    // large enough to hold all enum values, i.e. if you have more
than 256 values,
    // `uint16` will be used and so on.
    function getChoice() public view returns (ActionChoices) {
        return choice;
    }

    function getDefaultChoice() public pure returns (uint) {
        return uint(defaultChoice);
    }
}
```



PRECEDENCE
VALUE TYPES

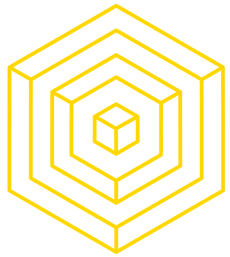
Precedence	Description	Operator
1	Postfix increment and decrement	`++` , `--`
	New expression	`new <typename>`
	Array subscripting	`<array>[<index>]`
	Member access	`<object>.<member>`
	Function-like call	`<func>(<args...>)`
	Parentheses	`(<statement>)`
2	Prefix increment and decrement	`++` , `--`
	Unary plus and minus	`+` , `-`
	Unary operations	`delete`
	Logical NOT	`!`
	Bitwise NOT	`~`
3	Exponentiation	`**`
4	Multiplication, division and modulo	`*` , `/` , `%`
5	Addition and subtraction	`+` , `-`
6	Bitwise shift operators	`<<` , `>>`
7	Bitwise AND	`&`
8	Bitwise XOR	`^`
9	Bitwise OR	`|`
10	Inequality operators	`<` , `>` , `<=` , `>=`
11	Equality operators	`==` , `!=`
12	Logical AND	`&&`
13	Logical OR	`||`
14	Ternary operator	`<conditional> ? <if-true> : <if-false>`
15	Assignment operators	`=` , `|=` , `^=` , `&=` , `<<=` , `>>=` , `+=` , `-=` , `*=` , `/=`
16	Comma operator	`,`



4.2 DATA STRUCTURES

DATA STRUCTURES
ARRAYS

```
// Arrays
bytes32[5] nicknames; // static array
bytes32[] names; // dynamic array
uint newLength = names.push("John"); // adding returns new length of the array
// Length
names.length; // get length
names.length = 1; // lengths can be set (for dynamic arrays in storage only)

// multidimensional array
uint x[][5]; // arr with 5 dynamic array elements (opp order of most languages)
```

DATA STRUCTURES
STRUCTS

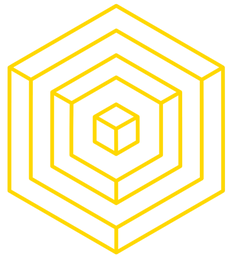
```
struct Bank {
    address owner;
    uint balance;
}
Bank b = Bank({
    owner: msg.sender,
    balance: 5
});
// or
Bank c = Bank(msg.sender, 5);

c.balance = 5; // set to new value
delete b;
// sets to initial value, set all variables in struct to 0, except mappings
```

DATA STRUCTURES
MAPPINGS

```
// Dictionaries (any type to any other type)
mapping (string => uint) public balances;
balances["charles"] = 1;
// balances["ada"] result is 0, all non-set key values return zeroes
// 'public' allows following from another contract
contractName.balances("charles"); // returns 1
// 'public' created a getter (but not setter) like the following:
function balances(string _account) returns (uint balance) {
    return balances[_account];
}
```

DATA STRUCTURES
MAPPINGS

```
// Nested mappings
mapping (address => mapping (address => uint)) public custodians;

// To delete
delete balances["John"];
delete balances; // sets all elements to 0
```



4.3 FUNCTIONS

Function basics

access classifier

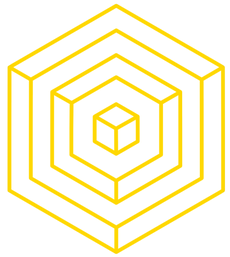
name **argType1 arg1, ...** **returnType (optional return var name)**

```
function withdraw(uint withdrawAmount) public returns (uint remainingBal) {
    require(withdrawAmount <= balances[msg.sender]);

    // Note the way we deduct the balance right away, before sending
    // Every .transfer/.send from this contract can call an external function
    // This may allow the caller to request an amount greater
    // than their balance using a recursive call
    // Aim to commit state before calling external functions, including .transfer/.send
    balances[msg.sender] -= withdrawAmount;

    // this automatically throws on a failure, which means the updated balance is reverted
    msg.sender.transfer(withdrawAmount);

    return balances[msg.sender];
}
```

- public - all can access
- external - Cannot be accessed internally, only externally
- internal - only this contract and contracts deriving from it can access
- private - can be accessed only from this contract

IMPORTANT:
Access classifiers determine who can USE your functions, but everyone can see them, since they will be deployed on a public blockchain.

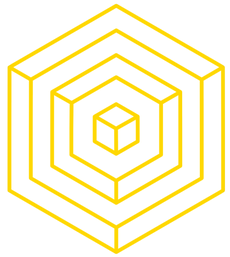
AUTHOR: NISHAAD NAVKAL



Payable functions

Only functions marked as payable can receive Ether
Non-payable functions with ether values will be routed to default function

```
function deposit() public payable returns (uint) {
    // Use 'require' to test user inputs, 'assert' for internal invariants
    // Here we are making sure that there isn't an overflow issue
    require((balances[msg.sender] + msg.value) >= balances[msg.sender]);

    balances[msg.sender] += msg.value;
    // no "this." or "self." required with state variable
    // all values set to data type's initial value by default

    LogDepositMade(msg.sender, msg.value); // fire event

    return balances[msg.sender];
}
```



Fallback function

- Invoked when a function is called which does not match any other contract functions
- Only one per contract
- No arguments, returns nothing
- Typically payable - enables contract to receive ether sent directly to it

Can be thought of as default behavior when the contract does not recognize the command

```
function() external payable {
    require(msg.value >= prize || msg.sender == owner);
    king.transfer(msg.value);
    king = msg.sender;
    prize = msg.value;
}
```

Constructors

- Create an instance of the contract with the given arguments
- Only one allowed, cannot be overloaded
- 2 implementations:
  - function [contractName] (arg1, arg2 ...)
  - constructor (arg1, arg2 …)

```
contract SimpleBank { // CapWords
    // Declare state variables outside function, persist through life of contract

    // dictionary that maps addresses to balances
    // always be careful about overflow attacks with numbers
    mapping (address => uint) private balances;

    // "private" means that other contracts can't directly query balances
    // but data is still viewable to other parties on blockchain

    address public owner;
    // 'public' makes externally readable (not writeable) by users or contracts

    // Events - publicize actions to external listeners
    event LogDepositMade(address accountAddress, uint amount);

    // Constructor, can receive one or many variables here; only one allowed
    function SimpleBank() public {
        // msg provides details about the message that's sent to the contract
        // msg.sender is contract caller (address of contract creator)
        owner = msg.sender;
    }
```

6.4 WORKSHOP TIME

Attendance/Vitamin